

06 · Geometry DSL 设计

2026-07-31

Contents

06 · Geometry DSL 设计1

1. 为什么 DSL 比图片更适合 LLM1

2. DSL 设计目标2

3. EBNF 语法2

4. DSL 示例3

5. 关键字全集3

6. 类型系统4

7. 解析器实现4

7.1 解析流程4

7.2 伪代码4

7.3 语义检查5

8. 序列化器实现5

8.1 序列化策略5

9. 反事实编辑5

10. DSL ↔ Graph 双向一致性6

11. 与 Prompt 的集成6

12. 设计路线小结6

06 · Geometry DSL 设计

本文档阐述 Geometry DSL 的设计动机、EBNF 语法、类型系统、解析器/序列化器实现，以及反事实编辑能力。DSL 是 Geometry Graph 的线性化、声明式、紧凑表达，专为 LLM 与 Solver 设计。

1. 为什么 DSL 比图片更适合 LLM

维度	图片输入	DSL 输入
信息精度	像素级，模糊	符号级，精确
LLM 解析成本	需 VLM，误差大	纯文本，准确
关系显式性	隐含，需推断	显式声明
可编辑/可注入	难	易（增删条件、反事实推理）
验证闭环	难	易（Solver 直接读 DSL）
Token 效率	图像 token 多	文本 token 少

DSL 将”看图”误差彻底隔离在感知层，LLM 拿到的是已经过 Verifier 验证的、符号化的、确定性的几何事实，从而能专注发挥推理与规划能力。这是本系统”感知-推理解耦”的关键载体。

2. DSL 设计目标

1. **声明式**：描述”是什么”而非”怎么画”。
 2. **紧凑**：Token 高效，适合 Prompt。
 3. **可读**：人/LLM/Solver 均可读。
 4. **可逆**：与 Geometry Graph 双向转换。
 5. **可扩展**：新对象/关系易加入。
 6. **机器可解析**：严格语法，无歧义。
-

3. EBNF 语法

```
Program      ::= ObjectSection RelationSection? GoalSection?

ObjectSection ::= "Objects:" ObjectDecl+
ObjectDecl   ::= "-" ObjectExpr
ObjectExpr   ::= PointDecl | LineDecl | SegmentDecl | RayDecl
               | CircleDecl | ArcDecl | EllipseDecl | PolygonDecl
PointDecl    ::= "Point(" Label ")" ":" Coords
LineDecl     ::= "Line(" Label "," PointRef "," PointRef ")"
SegmentDecl  ::= "Segment(" Label ")"
RayDecl      ::= "Ray(" Label "," PointRef "," Dir ")"
CircleDecl   ::= "Circle(" Label "," "r=" Number ")"
ArcDecl      ::= "Arc(" Label "," PointRef "," PointRef ")"
EllipseDecl  ::= "Ellipse(" Label "," "c=" Coords "," "a=" Number
               "," "b=" Number "," "theta=" Number ")"

RelationSection ::= "Relations:" RelationDecl+
RelationDecl    ::= "-" RelationExpr
RelationExpr    ::= "On(" PointRef "," ObjRef ")"
               | "Collinear(" PointRef ("," PointRef)+ ")"
               | "Intersect(" ObjRef "," ObjRef ")"
               | "Tangent(" ObjRef "," ObjRef ("," "at=" PointRef)? ")"
               | "Parallel(" ObjRef "," ObjRef ")"
               | "Perpendicular(" ObjRef "," ObjRef ")"
               | "Equal(" ObjRef "," ObjRef ")"
               | "Inside(" PointRef "," ObjRef ")"
               | "Concentric(" ObjRef "," ObjRef ")"
               | "Angle(" PointRef "," PointRef "," PointRef ")" "=" AngleVal
               | "SumDist(" PointRef "," PointRef "," PointRef ")" "=" Expr
               | ...

GoalSection    ::= "Goal:" GoalExpr
GoalExpr       ::= "Prove:" Proposition | "Solve:" Var | "Find:" Proposition
```

ObjRef ::= Label | ObjCall
ObjCall ::= ("Segment"|"Line"|"Circle"|"Ellipse") "(" Label ")"
PointRef ::= Label
Coords ::= "[" Number "," Number "]"
Dir ::= "[" Number "," Number "]"
Label ::= Letter+
Number ::= [0-9]+ ("." [0-9]+)?
AngleVal ::= Number "deg"
Proposition ::= ... (* 等式/不等式表达式 *)

4. DSL 示例

Objects:

- Point(A): [180.0, 84.5]
- Point(B): [260.0, 140.0]
- Point(O): [180.0, 160.0]
- Point(P): [243.2, 195.3]
- Circle(O, r=75.5)
- Segment(AB)
- Segment(OA)
- Ellipse(E, c=[200,180], a=90, b=50, theta=0.35)

Relations:

- On(A, Circle(O))
- Tangent(Segment(AB), Circle(O), at=A)
- Perpendicular(Segment(OA), Segment(AB))
- On(P, Ellipse(E))
- SumDist(P, F1, F2) = 180
- Equal(Segment(AC), Segment(BC))
- Parallel(Segment(DE), Segment(AB))
- Angle(BAC) = 60deg

Goal:

- **Prove:** Equal(Product(AB, AC), Product(AD, AE))

5. 关键字全集

类别	关键字
对象声明	Point, Line, Segment, Ray, Circle, Arc, Ellipse, Polygon, Triangle
位置关系	On, Inside, Outside, Collinear, Intersect
度量关系	Equal, Angle, Length, Parallel, Perpendicular

类别	关键字
圆关系	Tangent, Secant, Chord, Inscribed, Circumscribed, Concentric, Power
椭圆关系	Focus, SumDist, Eccentricity, Tangent(to ellipse)
目标	Prove, Solve, Find
函数	Product, Sum, Ratio, Sqrt, Sin, Cos, Tan

6. 类型系统

DSL 隐式类型系统:

类型	取值	出现位置
Point	标签	On/Inside/Angle 参数
LineObj	Line/Segment/Ray	Tangent/Parallel/Perpendicular 参数
Curve	Circle/Arc/Ellipse	On/Tangent 参数
Number	数值	半径/角度/坐标
Bool	true/false	Proposition
Expr	表达式	Goal/SumDist 右值

类型检查在解析阶段执行，类型不匹配报错（如 On(Circle, Point) 参数顺序错），保证语义正确。

7. 解析器实现

解析器将 DSL 文本解析为 AST，再转为 Geometry Graph。采用 lark 或 pyparsing 实现 EBNF。

7.1 解析流程

DSL 文本 —► Lexer/Parser —► AST —► SemanticCheck —► GeometryGraph

7.2 伪代码

```
from lark import Lark, Transformer

dsl_grammar = open("dsl_grammar.lark").read()
parser = Lark(dsl_grammar, start="program", parser="earley")

class DSLToGraph(Transformer):
    def program(self, items):
        g = GeometryGraph()
        for sec in items:
            if isinstance(sec, ObjectsSection):
                for obj in sec.objs:
                    g.add_node(obj.id, **obj.attrs)
            elif isinstance(sec, RelationsSection):
```

```

        for rel in sec.rels:
            g.add_edge(rel.src, rel.dst, rel=rel.type,
                        verified="true", attrs=rel.attrs)
        elif isinstance(sec, GoalSection):
            g.goal = sec.goal
    return g

def from_dsl(text: str) -> GeometryGraph:
    tree = parser.parse(text)
    return DSLToGraph().transform(tree)

```

7.3 语义检查

- **引用完整性**: 所有 PointRef/ObjRef 必须在 Objects 段声明。
- **类型一致性**: 关系参数类型匹配。
- **冗余检测**: 重复声明警告。

8. 序列化器实现

序列化器将 Geometry Graph 输出为 DSL 文本，供 LLM Prompt 使用。

```

def to_dsl(graph: GeometryGraph) -> str:
    lines = ["Objects:"]
    for n in graph.nodes:
        lines.append(f" - {format_object(graph, n)}")
    verified_edges = [e for e in graph.edges
                      if graph.edges[e].get("verified")=="true"]
    if verified_edges:
        lines.append("Relations:")
        for e in verified_edges:
            lines.append(f" - {format_relation(graph, e)}")
    if graph.goal:
        lines.append(f"Goal:\n - {format_goal(graph.goal)}")
    return "\n".join(lines)

```

8.1 序列化策略

- **仅输出已验证边**: verified=true 的关系才进 DSL，避免误导 LLM。
- **弱假设标注**: uncertain 关系以注释形式输出 (# uncertain: ...), 供 LLM 作线索。
- **紧凑模式**: 可选省略坐标，仅保留拓扑 (减少 token)，用于长上下文。

9. 反事实编辑

DSL 的可编辑性支持**反事实推理**与辅助构造:

操作	用途
删除关系	“若无此切线，结论是否成立？”
添加辅助线	LLM 构造辅助点/线，加入 DSL 重新求解
修改参数	“若半径改为 r2，结果如何？”
假设注入	LLM 提出假设关系，注入 DSL 供 Solver 验证

```
def inject_hypothesis(dsl: str, hyp: str) -> str:
    """LLM 在 DSL 末尾追加假设关系，供 Solver 验证。"""
    return dsl + f"\n# hypothesis\n - {hyp}"
```

反事实编辑是 LLM 进行探索性证明（ToT 搜索）的基础能力。

10. DSL ↔ Graph 双向一致性

- $\text{Graph} \rightarrow \text{DSL} \rightarrow \text{Graph}'$: 要求 $G = G'$ (同构)。
- 单元测试覆盖：对随机生成 Graph，往返转换后断言一致。
- 序列化器与解析器互为逆运算，保证 DSL 作为中间表示的可靠性。

11. 与 Prompt 的集成

DSL 作为 LLM Prompt 的 [Context] 段注入：

```
[Context]
# Geometry DSL
Objects:
- Point(A): [180.0, 84.5]
- Circle(0, r=75.5)
- Segment(AB)
Relations:
- On(A, Circle(0))
- Tangent(Segment(AB), Circle(0), at=A)
- Perpendicular(Segment(OA), Segment(AB))

# 题目
如图，AB 切圆 O 于 A，OA⊥AB...

[Task]
Goal:
- Prove: ...
```

DSL 紧凑、显式、可验证的特性，使 LLM 能稳定理解几何前提。

12. 设计路线小结

Geometry DSL 的设计路线为：“动机分析 (精度/token/可编辑) → EBNF 语法定义 → 类型系统 → 解析器 (文本 → Graph) → 序列化器 (Graph → 文本) → 双向一致性测试 → 反事实编辑能力 → Prompt 集成”。DSL 是感知与推理

之间的”通用语”，使 LLM 摆脱像素不确定性，在符号化几何事实上发挥推理能力。